

WEEK 11 — Unit 4: Maven Build Automation

Topic: Parent POM, Dependency Scopes, Transitive Dependencies, Version Conflicts, Dependency Management, Maven Plugins & Maven Wrapper Course: INT 332 – DevOps

Note 1: Parent POM, Dependency Scopes & Transitive Dependencies

Parent POM

What is a Parent POM?

In large projects, you often have multiple Maven modules — for example, a user-service, an order-service, and a payment-service, all part of the same application. Each of these is a separate Maven project with its own pom.xml. Without a parent POM, you would have to repeat the same dependency versions, plugin configurations, Java version settings, and encoding properties in every single module's pom.xml. This creates a maintenance nightmare — updating a dependency version means editing every module's POM individually.

A Parent POM solves this by acting as a central configuration file that all child modules inherit from. Common settings are defined once in the parent and automatically applied to all children.

Structure of a Multi-Module Maven Project

my-project/

├─ pom.xml ← Parent POM

├─ user-service/

| └─ pom.xml ← Child POM

├─ order-service/

| └─ pom.xml ← Child POM

└─ payment-service/

└─ pom.xml ← Child POM

Parent POM (pom.xml at root level)

```
<project>
```

```
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.example</groupId>
```

```
<artifactId>my-project</artifactId>

<version>1.0.0</version>

<packaging>pom</packaging>

<modules>

  <module>user-service</module>

  <module>order-service</module>

  <module>payment-service</module>

</modules>

<properties>

  <maven.compiler.source>17</maven.compiler.source>

  <maven.compiler.target>17</maven.compiler.target>

  <spring.version>3.1.0</spring.version>

</properties>

<dependencyManagement>

  <dependencies>

    <dependency>

      <groupId>org.springframework.boot</groupId>

      <artifactId>spring-boot-starter-web</artifactId>

      <version>${spring.version}</version>

    </dependency>

  </dependencies>

</dependencyManagement>

</project>
```

Child POM (e.g., user-service/pom.xml)

```
<project>

  <modelVersion>4.0.0</modelVersion>

  <parent>

    <groupId>com.example</groupId>

    <artifactId>my-project</artifactId>

    <version>1.0.0</version>

  </parent>

  <artifactId>user-service</artifactId>

  <dependencies>

    <dependency>

      <groupId>org.springframework.boot</groupId>

      <artifactId>spring-boot-starter-web</artifactId>

      <!-- No version needed — inherited from parent -->

    </dependency>

  </dependencies>

</project>
```

Key Points About Parent POM

- The parent POM uses `<packaging>pom</packaging>` — it does not produce a JAR or WAR itself
- The `<modules>` section lists all child modules by their directory name
- Running `mvn install` from the parent directory builds all modules in the correct order
- Child POMs inherit: properties, dependency versions (via `dependencyManagement`), plugin configurations, and build settings

- The Spring Boot Starter Parent is a widely used example of a parent POM — when you create a Spring Boot project, its POM inherits from spring-boot-starter-parent, which provides pre-configured versions for hundreds of common dependencies
-

Dependency Scope

Dependency scope controls two things:

- **When** the dependency is available during the build lifecycle
- **Whether** the dependency is included in the final packaged artifact

Maven has six dependency scopes:

1. compile (default) The dependency is available at all phases — compile, test, and runtime. It is included in the final JAR/WAR. Use this for libraries your application needs to run.

```
<dependency>
```

```
  <groupId>org.springframework.boot</groupId>
```

```
  <artifactId>spring-boot-starter-web</artifactId>
```

```
  <version>3.1.0</version>
```

```
  <scope>compile</scope> <!-- This is the default; can be omitted -->
```

```
</dependency>
```

2. test The dependency is only available during the test compilation and execution phases. It is not included in the final artifact. Use this for testing libraries like JUnit, Mockito, and AssertJ.

```
<dependency>
```

```
  <groupId>org.junit.jupiter</groupId>
```

```
  <artifactId>junit-jupiter</artifactId>
```

```
  <version>5.9.3</version>
```

```
  <scope>test</scope>
```

```
</dependency>
```

3. provided The dependency is needed for compilation but will be provided by the runtime environment (e.g., a servlet container like Tomcat). It is not included in the final WAR because Tomcat already has it. Use this for the Servlet API when building web applications.

```
<dependency>
```

```
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
```

4. runtime The dependency is not needed for compilation but is needed at runtime. It is included in the final artifact. Use this for JDBC drivers — your code compiles against the standard java.sql interfaces, but the actual driver is only needed at runtime.

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.6.0</version>
  <scope>runtime</scope>
</dependency>
```

5. system Similar to provided, but you specify the exact path to the JAR on your local filesystem. This is rarely used and considered bad practice because it breaks portability.

6. import Only valid inside <dependencyManagement>. Used to import the dependency management section from another POM, typically a Bill of Materials (BOM) POM.

Transitive Dependencies

What are Transitive Dependencies?

When your project depends on library A, and library A itself depends on library B, then library B is a **transitive dependency** of your project. Maven automatically resolves and downloads all transitive dependencies — you do not need to declare them yourself.

Example:

- Your project declares a dependency on spring-boot-starter-data-jpa
- That library internally depends on hibernate-core, spring-data-jpa, jakarta.persistence-api, and many others
- Maven downloads all of them automatically

Why This is Powerful

Without transitive dependency resolution, you would have to manually declare every library that every library you use also needs — potentially hundreds of entries in your POM for a Spring Boot project.

Viewing the Dependency Tree

To see the full tree of all direct and transitive dependencies:

```
mvn dependency:tree
```

This outputs a tree showing which dependency pulled in which other dependency, which is very useful for debugging version conflicts.
